

Week 6: BattleOps Assignments

Objective: Ship software at scale with airtight guardrails, signed artifacts, reproducible builds, policy-driven pipelines, and hardened internal platforms.

MONDAY: Enterprise CI/CD Security & Hardened Pipelines

Set up Sandbox Repo

- `mkdir battleops-week6 && cd battleops-week6`
- `gh repo create battleops-week6 --private`
- `git init && git add . && git commit -m "init."`
- `git push origin main`

LAB 1: Enable GitHub OIDC for AWS (No static AWS keys allowed)

1. Create IAM Role:`aws iam create-role --role-name GitHubOIDCDeployRole \`
`--assume-role-policy-document file://trust-policy.json`

LAB 2: Add Terraform Plan-Only Enforcement

`.github/workflows/ci.yml:`

Jobs:

Tf-plan:

runs-on: ubuntu-latest

Permissions:

id-token: write

contents: read

Steps:

- uses: actions/checkout@v3

- name: Configure AWS via OIDC

uses: aws-actions/configure-aws-credentials@v4

With:

role-to-assume: arn:aws:iam::<ACCOUNT>:role/GitHubOIDCDeployRole

aws-region: ap-south-1

- name: Terraform Init

run: terraform -chdir=infra init

- name: Terraform Plan

run: terraform -chdir=infra plan -out=tf.plan

LAB 3: Ephemeral Self-Hosted Runner (Docker Sandbox)

`docker run -d --name runner \`

- `-e RUNNER_SCOPE=repo \`
- `-e REPO_URL=https://github.com/<YOU>/battleops-week6 \`
- `-e RUNNER_TOKEN=<TOKEN> \`
- `ghcr.io/actions/runner:latest`

Validation: Push → Build must run inside ephemeral runner → tear itself down.



TUESDAY: SBOM + Signing + Attestations + Admission Control

Build Image: docker build -t battleops-app:1.0 .

LAB 1: Generate SBOM + Upload as Artifact

```
syft battleops-app:1.0 -o json > sbom.json
```

CI Step:- name: Generate SBOM

run: syft battleops-app:1.0 -o json > sbom.json

Upload to GitHub:- uses: actions/upload-artifact@v3

With:

name: sbom

path: sbom.json

LAB 2: Sign Image + Upload to Rekor Transparency Log

Generate keys: cosign generate-key-pair

Sign: cosign sign --key cosign.key battleops-app:1.0

Record to Rekor: cosign sign --key cosign.key --rekor-url https://rekor.sigstore.dev battleops-app:1.0

LAB 3: Enforce Admission: Only Signed Images Allowed

Install Kyverno policy: apiVersion: kyverno.io/v1

kind: ClusterPolicy

Metadata:

name: verify-image-signature

Spec:

validationFailureAction: enforce

Rules:

- name: require-signed-images

Match:

Resources:

kinds: ["Pod"]

verifyImages:

- image: "**"

key: "cosign.pub"

Test: kubectl run unsigned --image=nginx

Should be blocked.

kubectl run signed --image=battleops-app:1.0

WEDNESDAY: Multi-Stage GitOps, Sync Waves, Drift Enforcement

Create Three Repo Layers

- `gh repo create app-repo`
- `gh repo create env-repo`
- `gh repo create cluster-repo`
- **app-repo** → application manifests
- **env-repo** → environment overlays (dev/stage/prod)
- **cluster-repo** → ArgoCD apps & policies

Create an App with Sync Waves

App.yaml:

Metadata:

Annotations:

`argocd.argoproj.io/sync-wave: "1"`

Configmap.yaml:

Metadata:

Annotations:

`argocd.argoproj.io/sync-wave: "0"`

→ ConfigMap applies before Deployment.

LAB 2: Fail Sync via Health Check

Modify Deployment: `livenessProbe: exec: command: ["false"]`

ArgoCD should:

- Detect app unhealthy
- Block sync
- Auto-rollback (if enabled)

LAB 3: Enforce Drift Correction Every 3 Minutes

- `argocd app set my-app --self-heal`
- `argocd app set my-app --sync-policy automated`
- `argocd app set my-app --sync-option Prune=true`

Now run: `kubectl edit deploy my-app`

Should auto-correct within 180 seconds.

THURSDAY: Platform Engineering: Golden Paths + Scorecards + IDP Guardrails

Create Backstage Template Folder

`mkdir -p templates/golden-service/skeleton`

LAB 1: Build Golden Path Template

Template.yaml: apiVersion: [scaffolder.backstage.io/v1beta3](https://backstage.io/docs/scaffolder/scaffolder-api)

kind: Template

metadata: name: golden-service

Spec: owner: platform-team

Steps:

- id: scaffold

action: fetch:template

Input:

url: ./skeleton

Skeleton includes:

- Dockerfile
- sbom.sh
- GitHub Actions pipeline
- manifest.yml (for GitOps)

LAB 2: Scorecards to Enforce Best Practices

Scorecards.yaml:

Checks:

- id: sbom-present

description: SBOM must exist

type: file

File:

pattern: "sbom.json"

- id: pipeline-present

File: pattern: ".github/workflows/*.yaml"

- id: signed-images-only

type: script

Script: cmd: "cosign verify <IMAGE>"

Backstage UI will show: Not production-ready

Ready for deployment

LAB 3: Create Self-Service Infra Provisioner

provision.sh:

- `#!/bin/bash`
- `echo "Creating namespace $1"`
- `kubectl create ns $1`
- `kubectl apply -f templates/golden-service/manifests -n $1`
- Backstage → click “Create Service” → infra + app deploy automatically.



FRIDAY: Runtime Enforcement: Zero-Trust Kubernetes

Install Kyverno or Gatekeeper

kubectl apply -f

<https://github.com/kyverno/kyverno/releases/latest/download/install.yaml>

LAB 1: Enforce Non-Root

apiVersion: kyverno.io/v1

kind: ClusterPolicy

Metadata:

name: enforce-non-root

Spec:

validationFailureAction: enforce

Rules:

- match:

Resources:

kinds: ["Pod"]

Validate:

message: "Containers must not run as root"

Pattern: spec: containers:- securityContext: runAsNonRoot: true

Test:kubectl run bad --image=nginx --command -- sleep 100

LAB 2: Enforce Mandatory Resource Limits

validate: pattern: spec: containers:

- resources: limits: cpu: "?*" memory: "?*"

Test:kubectl apply -f pod-without-limits.yaml

LAB 3: Enforce Only Signed Images + Non-Root + Limits (Composite Policy)

Combine:

- signature verification
- non-root
- resource limits
- registry whitelist

This becomes your **production runtime policy pack**.

Validation: Deploy a pod missing ANY requirement → must be rejected.



THE DEVOPS WAR ROOM